

TABLE OF CONTENT

ABSTRACT

1. INTRODUCTION

1.1 PROJECT AIMS AND OBJECTIVES

1.2 BACKGROUND OF PROJECT

1.3 OPERATION ENVIRONMENT

2. SYSTEM ANALYSIS

2.1 SOFTWARE REQUIREMENT SPECIFICATION

2.2 EXISTING VS PROPOSED

2.3 SOFTWARE TOOL USED

3. SYSTEM DESIGN

3.1 TABLE DESIGN

3.2 DATA FLOW DIAGRAMS

4. SYSTEM IMPLEMENTATION

4.1 MODULE DESCRIPTION

4.2 SCREEN SHOTS

5. SYSTEM TESTING

5.1 UNIT TESTING

5.2 INTEGRATION TESTING

6. CONCLUSION & FUTURE SCOPE

7. REFERENCES

ABSTRACT

Online Library Management System is a system which maintains the information about the books present in the library, their authors, the members of library to whom books are issued, library staff and all. This is very difficult to organize manually. Maintenance of all this information manually is a very complex task. Owing to the advancement of technology, organization of an Online Library becomes much simple. The Online Library Management has been designed to computerize and automate the operations performed over the information about the members, book issues and returns and all other operations. This computerization of library helps in many instances of its maintenances. It reduces the workload of management as most of the manual work done is reduced.

INTRODUCTION

1.1 PROJECT AIMS AND OBJECTIVES

The project aims and objectives that will be achieved after completion of this project are discussed in this subchapter. The aims and objectives are as follows:

- Online book reading.
- Facility to download required book.
- User login page where only authorised users are allowed to see and read all the available books.
- An Admin login page where admin can add books, page sources.
- Open link for Learning Websites.

1.2 BACKGROUND OF PROJECT

E-Library Management System is an application which refers to library systems which are generally small or medium in size. It is used by librarian to manage the library using a computerized system where he/she can add new books, and Page sources. Books and student maintenance modules are also included in this system which would keep track of the students using the library and also a detailed description about the books a library contains. With this computerized system there will be no loss of book record or member record which generally happens when a non-computerized system is used. All these modules are able to help librarian to manage the library with more convenience and in a more efficient way as compared to library systems which are not computerized.

1.3 OPERATION ENVIRONMENT

Processor -> intel core i5 processor or better performance

Operating system -> Windows 11

Memory -> 8gb ram or more

Hard disk space -> Minimum 3 GB for database usage for future

Database -> MySQL

SYSTEM ANALYSIS

2.1 SOFTWARE REQUIREMENT SPECIFICATION

PRODUCT DESCRIPTION:

Library Management System is a computerized system which helps user(librarian) to manage the library daily activity in electronic format. It reduces the risk of paper work such as file lost, file damaged and time consuming. It can help user to manage the transaction or record more effectively and timesaving.

PROBLEM STATEMENT:

The problem occurred before having computerized system includes:

- File lost When computerized system is not implemented file is always lost because of human environment. Sometimes due to some human error there may be a loss of records.
- File damaged When a computerized system is not there file is always lost due to some accident like spilling of water by some member on file accidentally. Besides some natural disaster like floods or fires may also damage the files.
- Difficult to search record When there is no computerized system there is always a difficulty in searching of records if the records are large in number.
- Space consuming After the number of records become large the space for physical storage of file and records also increases if no computerized system is implemented.
- Cost consuming as there is no computerized system the to add each record paper will be needed which will increase the cost for the management of library.

SYSTEM OBJECTIVES:

- Improvement in control and performance- The system is developed to cope up with the current issues and problems of library. The system can add user, validate user and is also bug free.
- Save cost- After computerized system is implemented less human force will be required to maintain the library thus reducing the overall cost.
- Save time- Librarian is able to search record by using few clicks of mouse and few search keywords thus saving his valuable time.
- Option of online Notice board- Librarian will be able to provide a detailed description of workshops going in the college as well as in nearby colleges.
- Lecture Notes- Admin has a facility to upload and edit lectures notes in a pdf file or, link of the materials and books.

SYSTEM REQUIREMENTS

A.NON FUNCTIONAL REQUIREMENTS

1.Product Requirement -

I. Efficiency Requirement - When a library management system will be implemented librarian and user will easily access library books and book transaction will be very faster.

II. Reliability Requirement - The system should accurately perform member registration, member validation, report generation, book reading.

III. Usability Requirement - The system is designed for a user-friendly environment so that student and staff of library can perform the various tasks easily and in an effective way.

2.Organizational Requirement -

I. Implementation Requirement - In implementing whole system it uses html in front end with spring boot i.e. Java which will be used for database connectivity and the backend, the database part is developed using MySQL.

II. Delivery Requirement - The whole system is expected to be delivered in two months of time with a weekly evaluation by the project guide.

B. FUNCTIONAL REQUIREMENTS

1.1 User Login - This feature used by the user to login into system. They are required to enter user id and password before they are allowed to enter the system. The user id and password will be verified and if invalid id is their user is not allowed to enter the system.

Functional requirements

- Unique user id should be used when they register.
- The system must only allow user with valid id and password to enter the system.
- The system performs authorization process which decides what user level can access to.
- The user must be able to logout after they finished using system.

1.2 Register New User- This feature can be performed by all users to register new user to create account.

Functional requirements

- System must be able to verify information.
- System must be able to delete information if information is wrong.

1.3 Register New Book- This feature allows an authorised admin to add new books to the library.

Functional requirements

- System must be able to verify information
- System must be able to enter number of copies into table.
- System must not be able to allow two books having same book id.

1.5 Edit Existing Book- This feature allows an authorised admin to edit existing books' name, author, category and the source of the book.

Functional requirements

-System must be able to update the details of the existing books, that are edited into the book list.

1.6 Delete Existing Book- This feature allows an authorised admin to delete existing books.

- System must be able to delete the details of the existing books' id.

SOFTWARE REQUIREMENTS

- Operating system- Windows 11 is used as the operating system as it is the latest available version and supports more features and is more user friendly.
- Database MySQL- MySQL is used as database as it easy to maintain and retrieve records by simple queries which are in English language which are easy to understand and easy to write.
- Development tools and Programming language- Java is used with Spring Boot framework in the back end and HTML is used to write the whole frontend code and develop webpages with CSS, JavaScript for styling work. We can implement the front end using Angular to make it easily accessible one paged application.

HARDWARE REQUIREMENTS

Intel core i5 12th generation is used as a processor because it is faster than other processors and provide reliability and stability and we can run our pc for long time. By using this processor, we can keep on developing our project without any worries.

Ram 8 GB is used as it will provide fast reading and writing capabilities and will in turn support in processing.

2.2 EXISTING VS PROPOSED

Existing System:

- Early days Libraries are managed manually. It required lot of time to record or to retrieve the details. The employees who have to record the details must perform their job very carefully. Even a small mistake would create a lot of problems. Security of information is very less. Report generations of all the information is very tough task.
- Maintenance of Library catalogue and arrangement of the books to the catalogue is very complex task. In addition to its maintenance of member details, issue dates and return dates etc. manually is a complex task.
- All the operations must be performed in perfect manner for the maintenance of the library without any degradation which may finally result in the failure of the entire system.

Proposed System:

To solve the inconveniences as mentioned in the existing system, an Online Library is proposed. The proposed system contains the following features:

- The students will register them through Online
- Individually each member will have his account through which he can access the information he needs.
- Book details like authors, number of copies totally maintained by library, present available number of books, reference books, non-reference books etc. all this information can be made handy.
- Regarding the members designation, number of books was issued.
- Issue dates and returns of each member is maintained separately and fine charged if there is any delay in returning the book.
- Administrator can add, update the books.
- Time consuming is low, gives accurate results, reliability can be improved with the help of security.

2.3 SOFTWARE TOOLS USED

The whole Project is divided in two parts the front end and the back end.

A.Front End

The front end is designed using HTML and Bootstrap, CSS and JavaScript.

HTML - HTML or Hyper Text Markup Language is the main markup language for creating web pages and other information that can be displayed in a web browser. HTML is written in the form of HTML elements consisting of tags enclosed in angle brackets, within the web page content. The purpose of a web browser is to read HTML documents and compose them into visible or audible web pages. HTML allows images and objects to be embedded and can be used to create interactive forms. It provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes and other items. It can embed scripts written in languages such as JavaScript which affect the behaviour of HTML web pages.

Bootstrap - Bootstrap is a free, open source front-end development framework for the creation of websites and web apps. Designed to enable responsive development of mobile-first websites, Bootstrap provides a collection of syntax for template designs.

Bootstrap makes responsive web design a reality. It makes it possible for a web page or app to detect the visitor's screen size and orientation and automatically adapt the display accordingly. The mobile-first approach assumes smartphones, tablets and task-specific mobile apps are employees' primary tools for getting work done. Bootstrap addresses the requirements of those technologies in design and includes UI components, layouts, JavaScript tools and the implementation framework. The software is available precompiled or as source code.

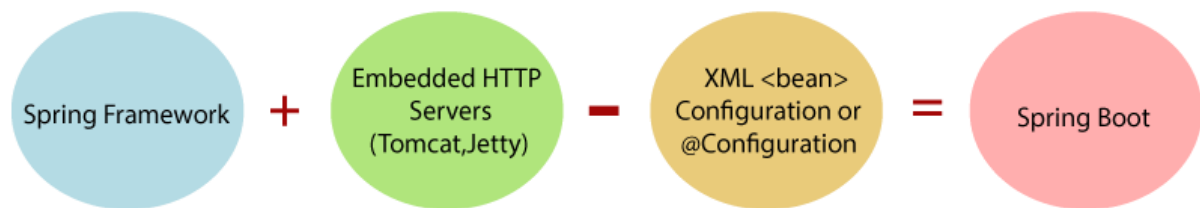
CSS - Cascading Style Sheets (CSS) is a style sheet language used for describing the look and formatting of a document written in a markup language. While most often used to style web pages and interfaces written in HTML and XHTML, the language can be applied to any kind of XML document, including plain XML, SVG and XUL. CSS is a cornerstone specification of the web and almost all web pages use CSS style sheets to describe their presentation. CSS is designed primarily to enable the separation of document content from document presentation, including elements such as the layout, colours and fonts. CSS can also allow the same markup page to be presented in different styles for different rendering methods, such as on-screen, in print, by voice and on Braille-based, tactile devices. It can also be used to allow the web page to display differently depending on the screen size or device on which it is being viewed.

JavaScript - JavaScript (JS) is a dynamic computer programming language. It is most commonly used as part of web browsers, whose implementations allow client-side scripts to interact with the user, control the browser, communicate asynchronously, and alter the document content that is displayed. It is also being used in server-side programming, game development and the creation of desktop and mobile applications. JavaScript is a prototype-based scripting language with dynamic typing and has first-class functions. It is a multiparadigm language, supporting object-oriented, imperative, and functional programming styles. The application of JavaScript to use outside of web pages—for example, in PDF documents, site-specific browsers, and desktop widgets—is also significant. Newer and faster JavaScript VMs and platforms built upon them (notably Node.js) have also increased the popularity of JavaScript for server-side web applications. On the client side, JavaScript was traditionally implemented as an interpreted language but just-in-time compilation is now performed by recent (post-2012) browsers.

B.Back End

Spring Boot - Spring Boot is a project that is built on the top of the Spring Framework. It provides an easier and faster way to set up, configure, and run both simple and web-based applications.

It is a Spring module providing the RAD (*Rapid Application Development*) feature to the Spring Framework. It is used to create a stand-alone Spring-based application that you can just run because it needs minimal Spring configuration.



In short, Spring Boot is the combination of Spring Framework and Embedded Servers.

In Spring Boot, there is no requirement for XML configuration (deployment descriptor). It uses convention over configuration software design paradigm that means it decreases the effort of the developer.

We can use Spring STS IDE or Spring Initializr to develop Spring Boot Java applications.

We should use Spring Boot Framework because:

- The dependency injection approach is used in Spring Boot.
- It contains powerful database transaction management capabilities.
- It simplifies integration with other Java frameworks like JPA/Hibernate ORM, Struts etc.
- It reduces the cost and development time of the application.

Spring Boot Version Used - 3.0.5

Database

MYSQL – MySQL is (as of July 2013) the world's second most widely used open-source relational database management system (RDBMS). The MySQL development project has made its source code available under the terms of the GNU General Public License, as well as under a variety of proprietary agreements. MySQL is a popular choice of database for use in web applications. Free-software-open-source projects that require a full-featured database management system often use MySQL. For commercial use, several paid editions are available, and offer additional functionality. Applications which use MySQL databases include: TYPO3, MODx, Joomla, WordPress, phpBB, MyBB, Drupal and other software. MySQL is also used in many high-profile, large-scale websites, including Wikipedia, Google (though not for searches), Facebook, Twitter, Flickr, and YouTube.

MySQL Version Used – 8.0.32

SYSTEM DESIGN

3.1 TABLE DESIGN

Entity Relationship Diagram

book_lms	
bid	BIGINT
bauthor	VARCHAR(255)
bcategory	VARCHAR(255)
blink	VARCHAR(255)
bname	VARCHAR(255)
Indexes	

user_details	
email	VARCHAR(255)
fullname	VARCHAR(255)
password	VARCHAR(255)
role	VARCHAR(255)
Indexes	

Tables to maintain information

- Book (book_lms) table from database

The screenshot displays a database management interface. At the top, a query editor shows the following SQL code:

```
11
12
13
14
15 • use mydb;
16 • select * from book_lms;
```

Below the query editor, a 'Result Grid' shows the data returned by the query. The grid has five columns: bid, bauthor, bcategory, blink, and bname. The data is as follows:

	bid	bauthor	bcategory	blink	bname
1	William Stallings	Computer Science and Engineering	http://uru.ac.in/uruonlineLibrary/Cyber_Securit...	Cryptography and Network Security	
2	P.T. Joseph	Computer Science and Engineering	https://www.drmishikantjha.com/booksCollectio...	E-Commerce	
6	Bcd	Comics		Abc	
10	vbc	Horror		vbi	
*	NULL	NULL	NULL	NULL	NULL

At the bottom, an 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2	00:54:03	show tables	14 row(s) returned	0.000 sec / 0.000 sec
3	00:54:03	select * from product LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
4	00:54:03	select * from book_lms LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

➤ User (user_details) table from database

The screenshot displays a database management interface. At the top, a query editor shows the following SQL code:

```
11
12
13
14
15 • use mydb;
16 • select * from user_details;
```

Below the query editor, the 'Result Grid' shows the results of the query. The grid has four columns: email, fullname, password, and role. The data is as follows:

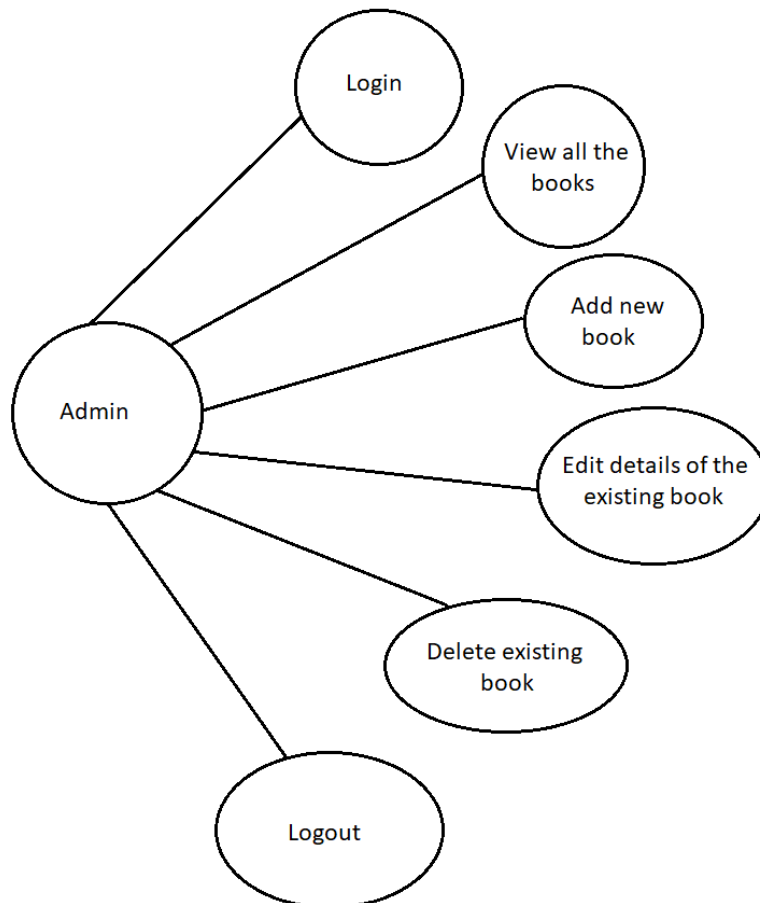
email	fullname	password	role
admin@gmail.com	admin	\$2a\$10\$/WZ8KVbcyUVspjJC2ni1c.naB6ClaoIsb...	ROLE_ADMIN
sayan@gmail.com	Sayan Nandi	\$2a\$10\$Grmb/8vpJmTXR4Onse.XPu83P3xAZjE...	ROLE_USER

At the bottom, the 'Action Output' pane shows the execution log:

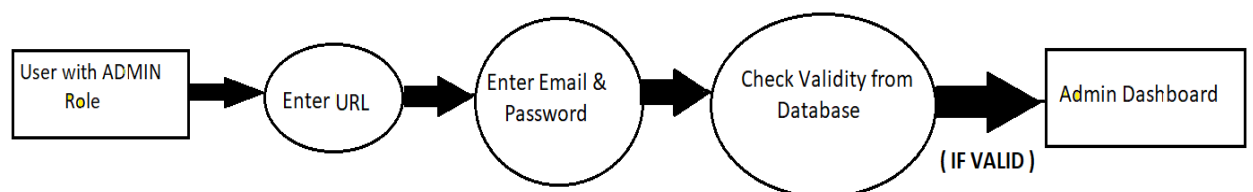
#	Time	Action	Message	Duration / Fetch
7	00:55:48	select * from product LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
8	00:55:48	use mydb	0 row(s) affected	0.000 sec
9	00:55:48	select * from user_details LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

3.2 DATA FLOW DIAGRAMS

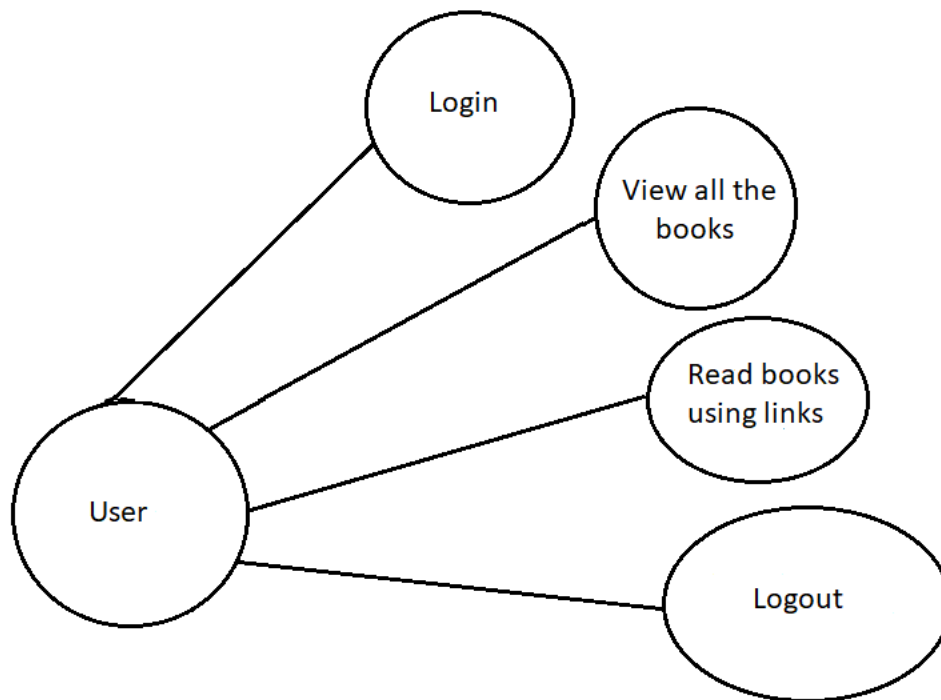
1.DFD for “ADMIN” user



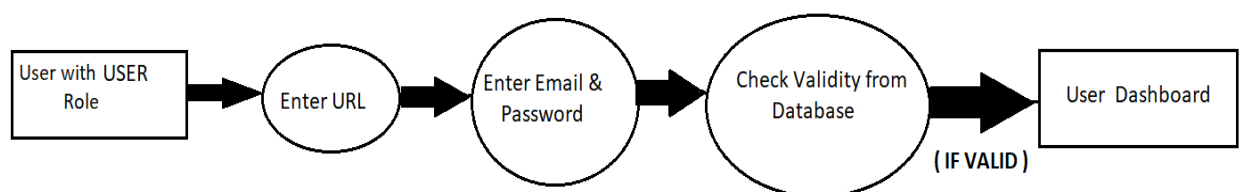
2.DFD for user login with role “ADMIN”



3.DFD for “USER” role

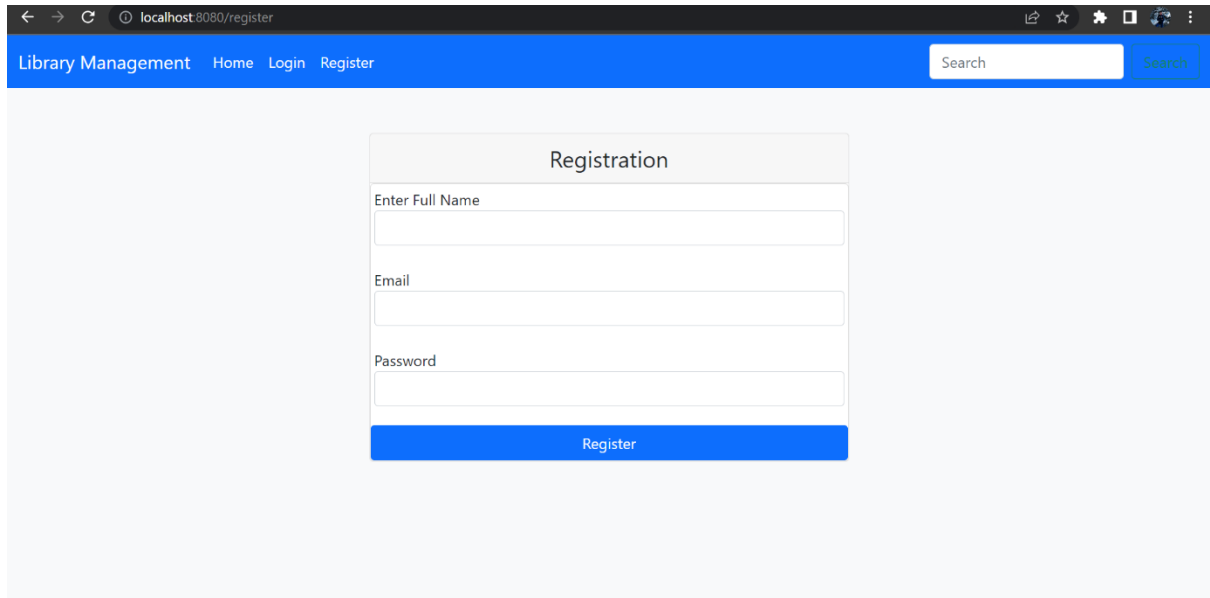


4.DFD for user login with role “USER”



SYSTEM IMPLEMENTATION

1.Registration Page



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/register'. The browser's address bar includes navigation icons (back, forward, refresh) and a search icon. The page has a blue header bar with the text 'Library Management' and navigation links 'Home', 'Login', and 'Register'. A search bar with the placeholder text 'Search' and a 'Search' button is located on the right side of the header. The main content area is light gray and contains a white registration form. The form has a title 'Registration' and three input fields labeled 'Enter Full Name', 'Email', and 'Password'. A blue 'Register' button is at the bottom of the form.

Registration

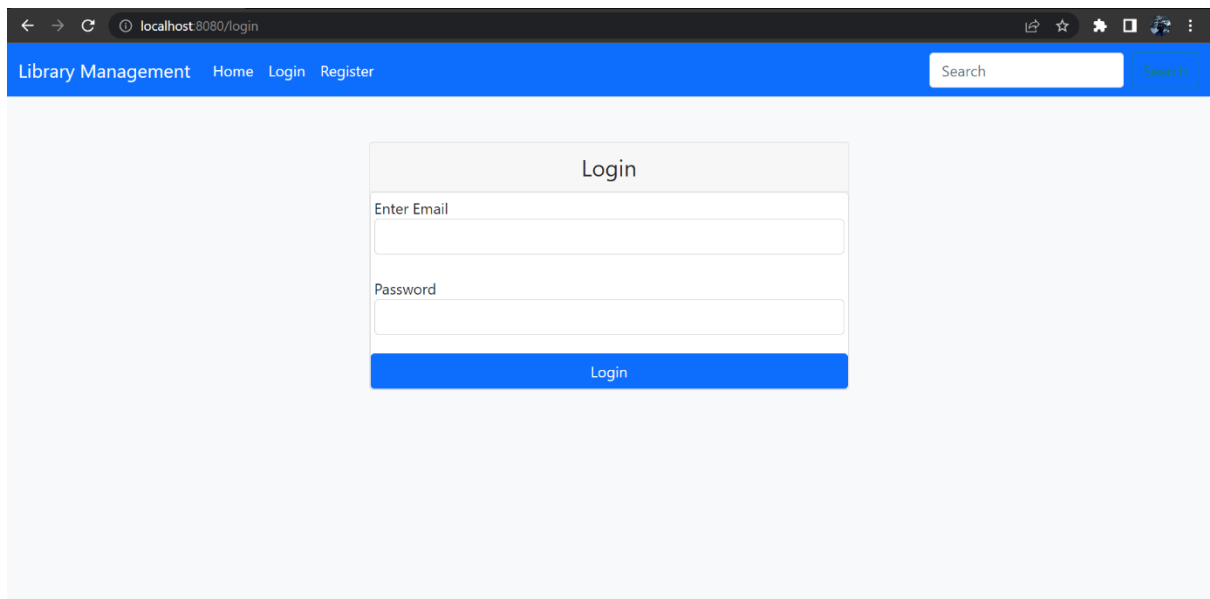
Enter Full Name

Email

Password

Register

2.Login Page



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/login'. The browser's address bar includes navigation icons (back, forward, refresh) and a search icon. The page has a blue header bar with the text 'Library Management' and navigation links 'Home', 'Login', and 'Register'. A search bar with the placeholder text 'Search' and a 'Search' button is located on the right side of the header. The main content area is light gray and contains a white login form. The form has a title 'Login' and two input fields labeled 'Enter Email' and 'Password'. A blue 'Login' button is at the bottom of the form.

Login

Enter Email

Password

Login

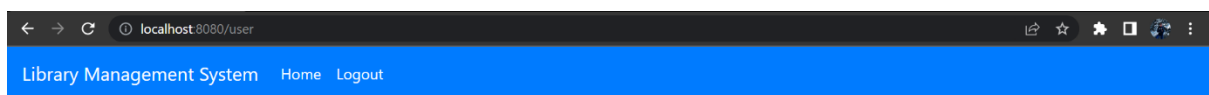
3.Admin Dashboard



Book List

Book Id	Book Name	Author Name	Book Category	Action
1	Cryptography and Network Security	William Stallings	Computer Science and Engineering	Edit Delete
2	E-Commerce	P.T. Joseph	Computer Science and Engineering	Edit Delete
6	Abc	Bcd	Comics	Edit Delete
10	vbi	vbc	Horror	Edit Delete

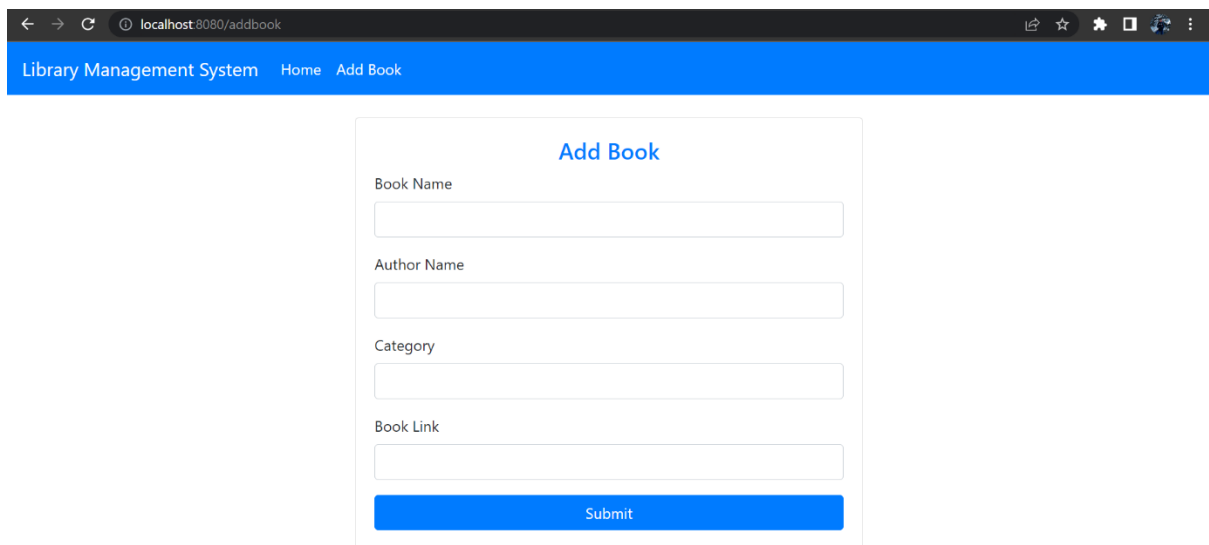
4.User Dashboard



Book List

Book Id	Book Name	Author Name	Book Category	Book Link
1	Cryptography and Network Security	William Stallings	Computer Science and Engineering	Link
2	E-Commerce	P.T. Joseph	Computer Science and Engineering	Link
6	Abc	Bcd	Comics	Link
10	vbi	vbc	Horror	Link

5.Add Book Page



localhost:8080/addbook

Library Management System Home Add Book

Add Book

Book Name

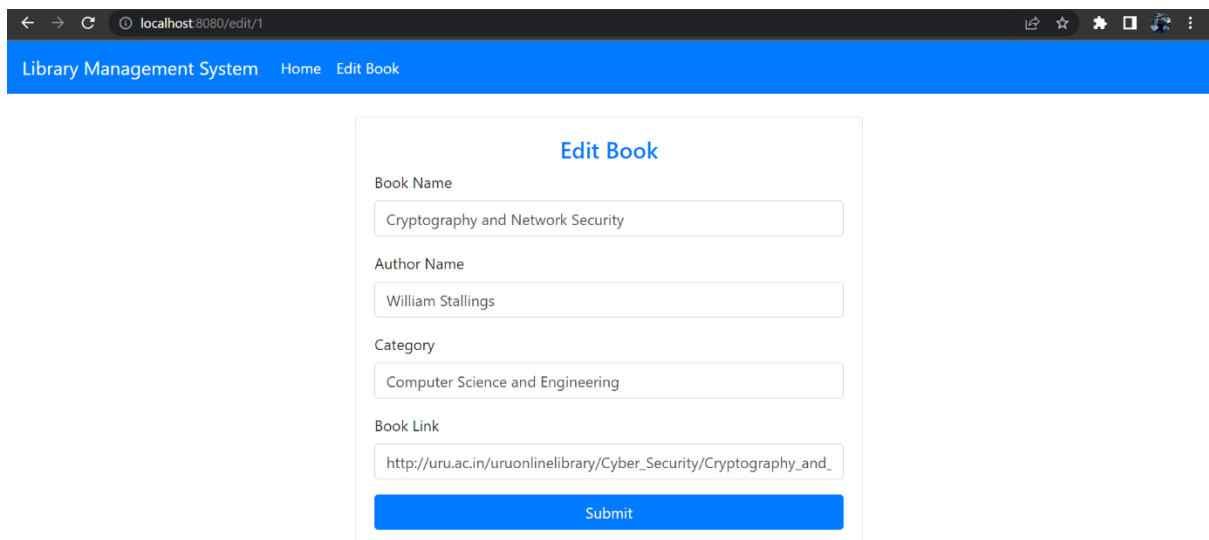
Author Name

Category

Book Link

Submit

6.Edit Book Page



localhost:8080/edit/1

Library Management System Home Edit Book

Edit Book

Book Name

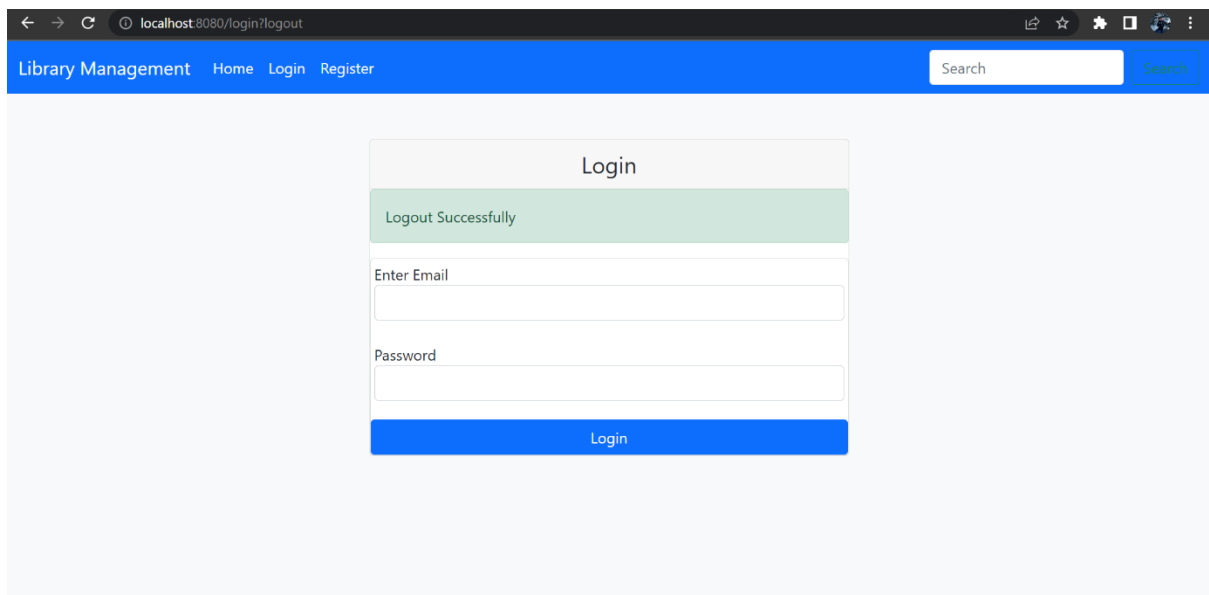
Author Name

Category

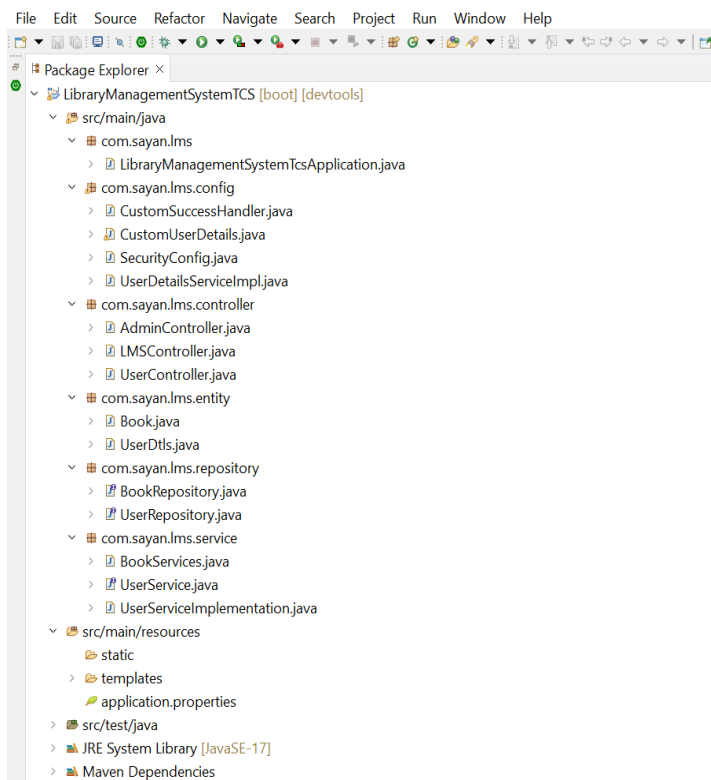
Book Link

Submit

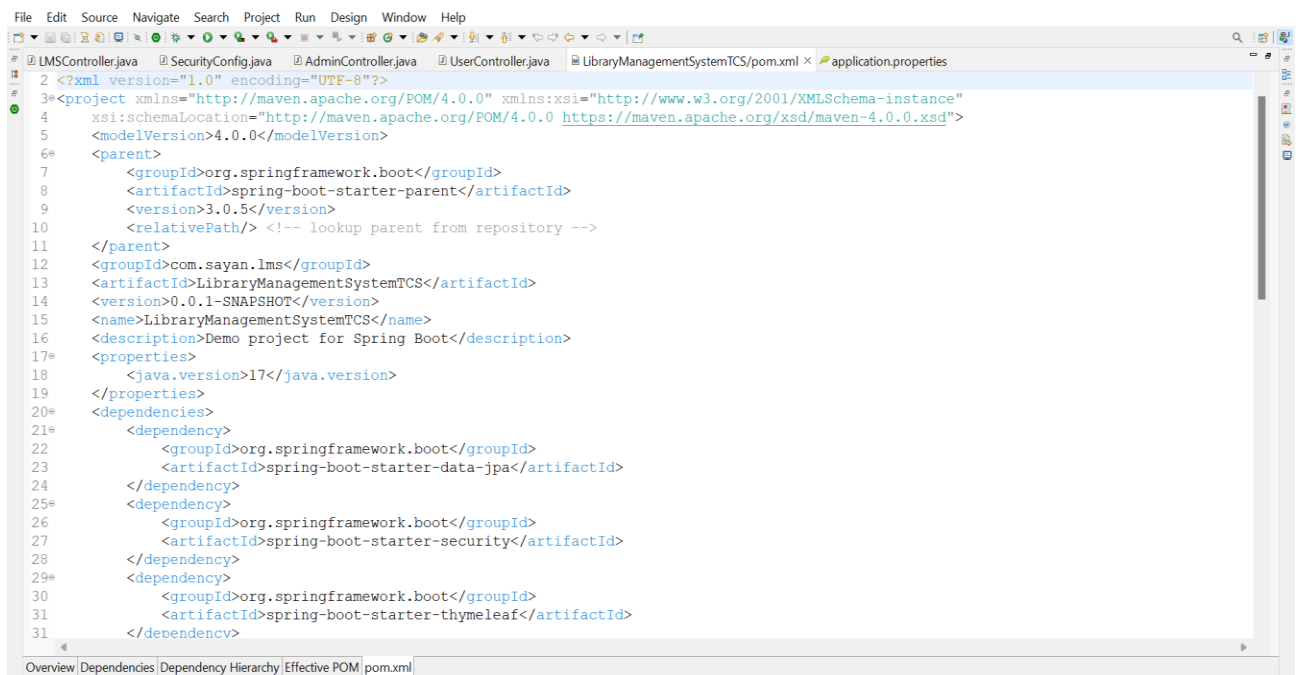
7.After Successfully Logging out



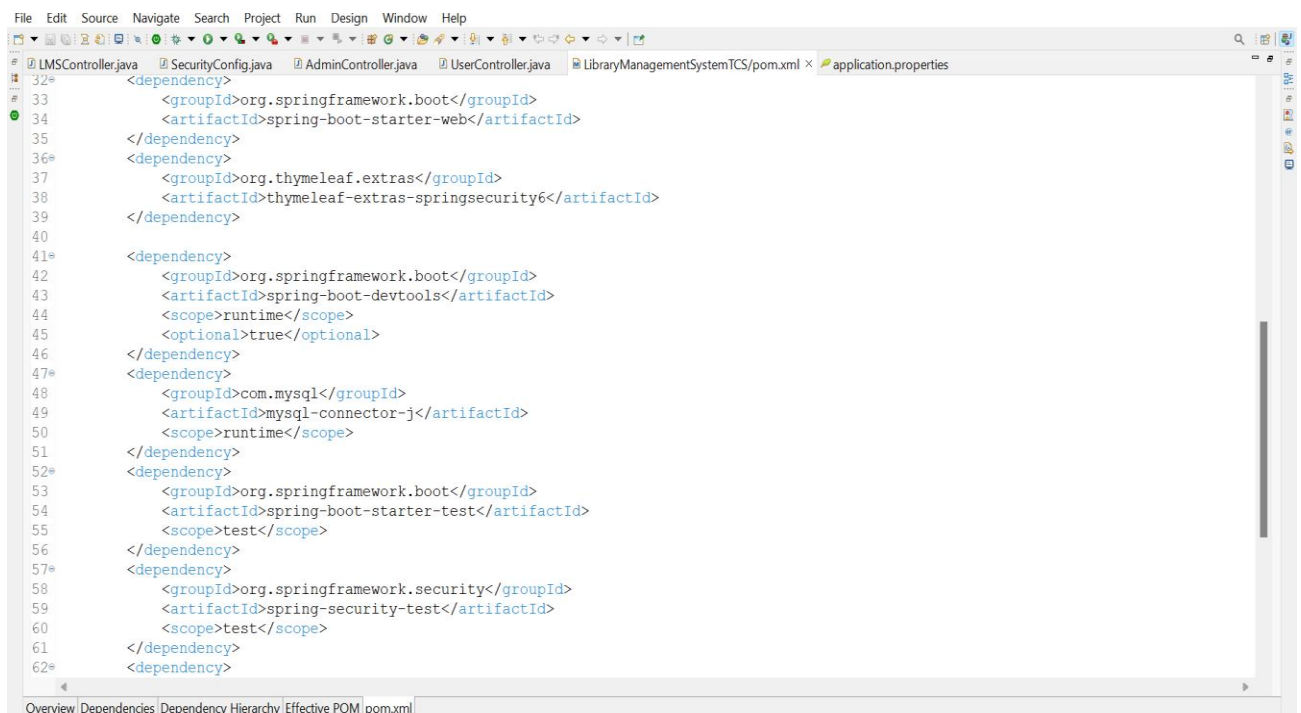
File Structure



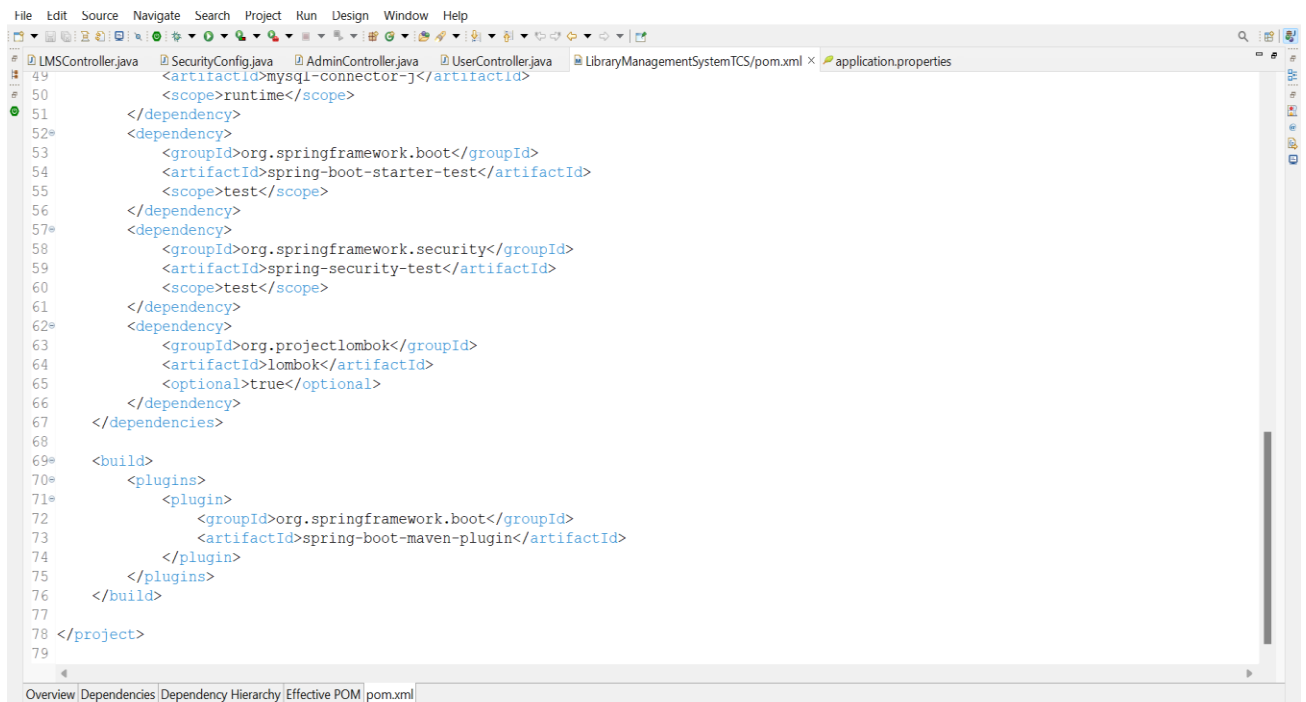
pom.xml file



```
2 <?xml version="1.0" encoding="UTF-8"?>
3 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6     <parent>
7         <groupId>org.springframework.boot</groupId>
8         <artifactId>spring-boot-starter-parent</artifactId>
9         <version>3.0.5</version>
10        <relativePath/> <!-- lookup parent from repository -->
11    </parent>
12    <groupId>com.sayan.lms</groupId>
13    <artifactId>LibraryManagementSystemTCS</artifactId>
14    <version>0.0.1-SNAPSHOT</version>
15    <name>LibraryManagementSystemTCS</name>
16    <description>Demo project for Spring Boot</description>
17    <properties>
18        <java.version>17</java.version>
19    </properties>
20    <dependencies>
21        <dependency>
22            <groupId>org.springframework.boot</groupId>
23            <artifactId>spring-boot-starter-data-jpa</artifactId>
24        </dependency>
25        <dependency>
26            <groupId>org.springframework.boot</groupId>
27            <artifactId>spring-boot-starter-security</artifactId>
28        </dependency>
29        <dependency>
30            <groupId>org.springframework.boot</groupId>
31            <artifactId>spring-boot-starter-thymeleaf</artifactId>
32        </dependency>
```

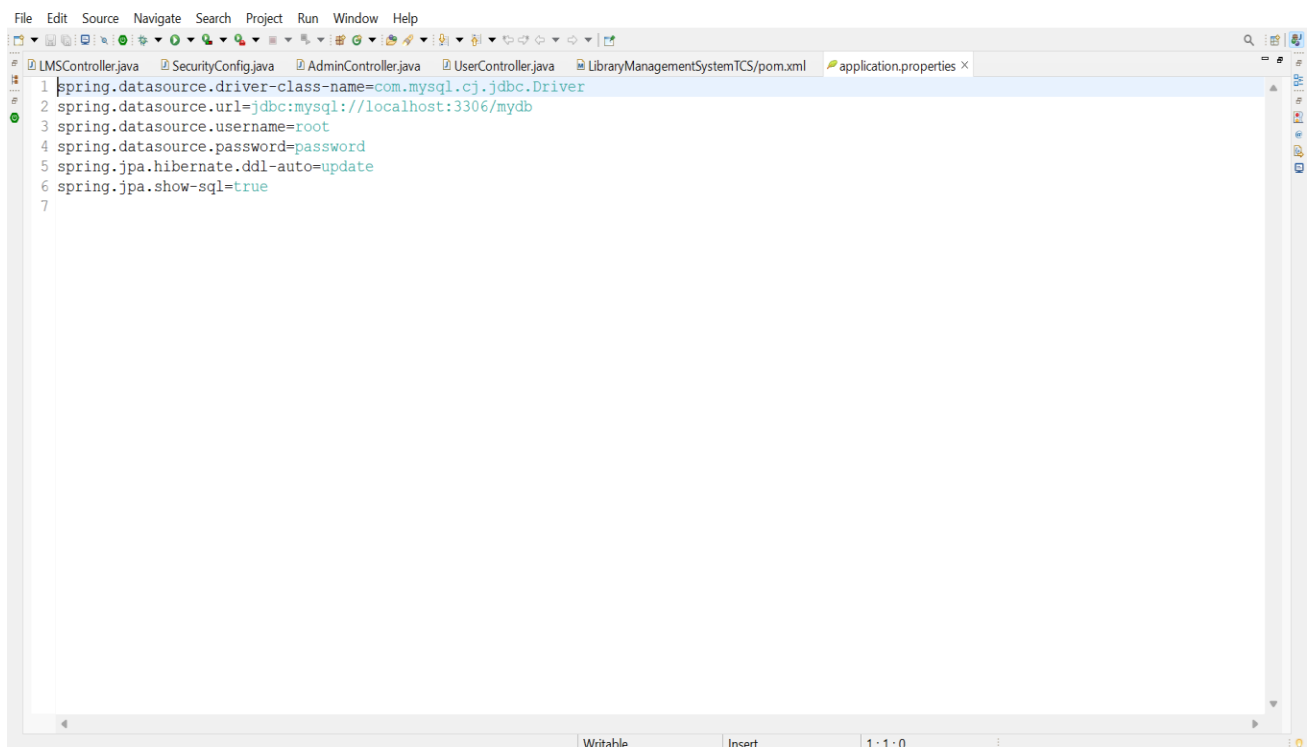


```
33    <dependency>
34        <groupId>org.springframework.boot</groupId>
35        <artifactId>spring-boot-starter-web</artifactId>
36    </dependency>
37    <dependency>
38        <groupId>org.thymeleaf.extras</groupId>
39        <artifactId>thymeleaf-extras-springsecurity6</artifactId>
40    </dependency>
41    <dependency>
42        <groupId>org.springframework.boot</groupId>
43        <artifactId>spring-boot-devtools</artifactId>
44        <scope>runtime</scope>
45        <optional>true</optional>
46    </dependency>
47    <dependency>
48        <groupId>com.mysql</groupId>
49        <artifactId>mysql-connector-j</artifactId>
50        <scope>runtime</scope>
51    </dependency>
52    <dependency>
53        <groupId>org.springframework.boot</groupId>
54        <artifactId>spring-boot-starter-test</artifactId>
55        <scope>test</scope>
56    </dependency>
57    <dependency>
58        <groupId>org.springframework.security</groupId>
59        <artifactId>spring-security-test</artifactId>
60        <scope>test</scope>
61    </dependency>
62    <dependency>
```



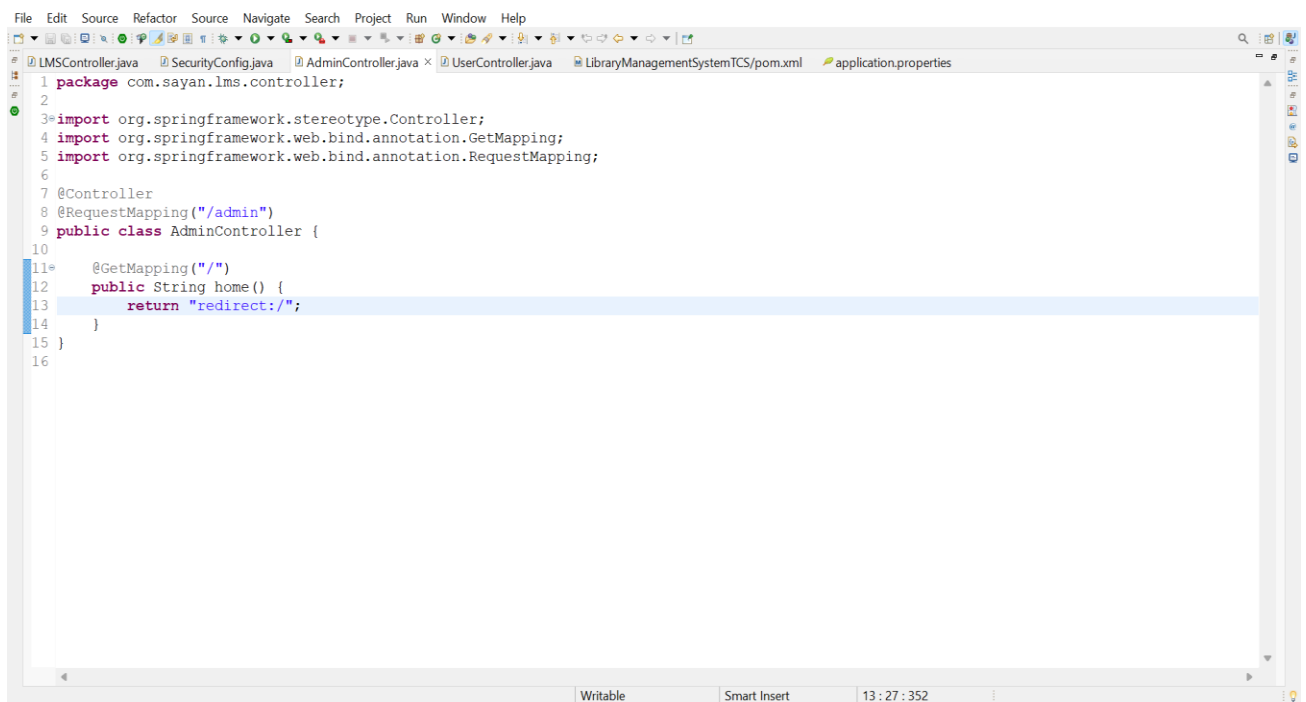
```
49 <artifactId>mysql-connector-j</artifactId>
50 <scope>runtime</scope>
51 </dependency>
52 <dependency>
53 <groupId>org.springframework.boot</groupId>
54 <artifactId>spring-boot-starter-test</artifactId>
55 <scope>test</scope>
56 </dependency>
57 <dependency>
58 <groupId>org.springframework.security</groupId>
59 <artifactId>spring-security-test</artifactId>
60 <scope>test</scope>
61 </dependency>
62 <dependency>
63 <groupId>org.projectlombok</groupId>
64 <artifactId>lombok</artifactId>
65 <optional>true</optional>
66 </dependency>
67 </dependencies>
68
69 <build>
70 <plugins>
71 <plugin>
72 <groupId>org.springframework.boot</groupId>
73 <artifactId>spring-boot-maven-plugin</artifactId>
74 </plugin>
75 </plugins>
76 </build>
77
78 </project>
79
```

application.properties file



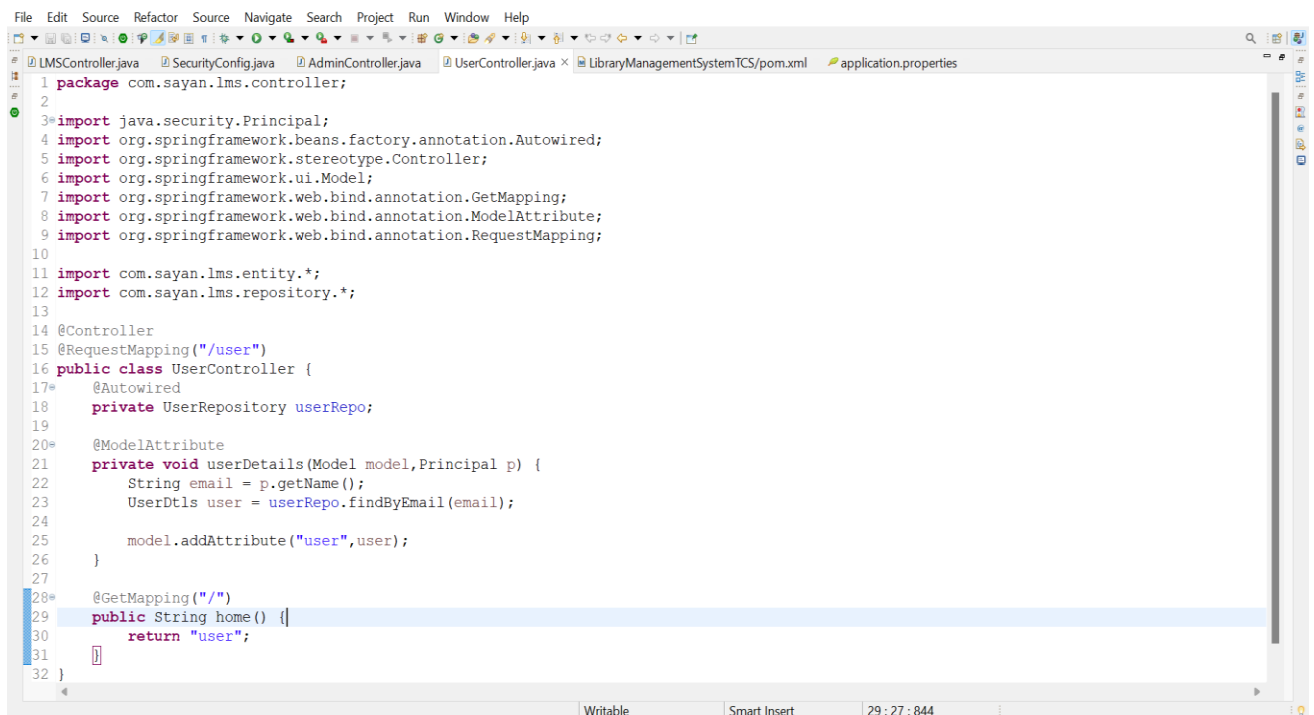
```
1 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
2 spring.datasource.url=jdbc:mysql://localhost:3306/mydb
3 spring.datasource.username=root
4 spring.datasource.password=password
5 spring.jpa.hibernate.ddl-auto=update
6 spring.jpa.show-sql=true
7
```

Admin Controller class



```
File Edit Source Refactor Source Navigate Search Project Run Window Help
1 package com.sayan.lms.controller;
2
3 import org.springframework.stereotype.Controller;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RequestMapping;
6
7 @Controller
8 @RequestMapping("/admin")
9 public class AdminController {
10
11     @GetMapping("/")
12     public String home() {
13         return "redirect:/";
14     }
15 }
16
```

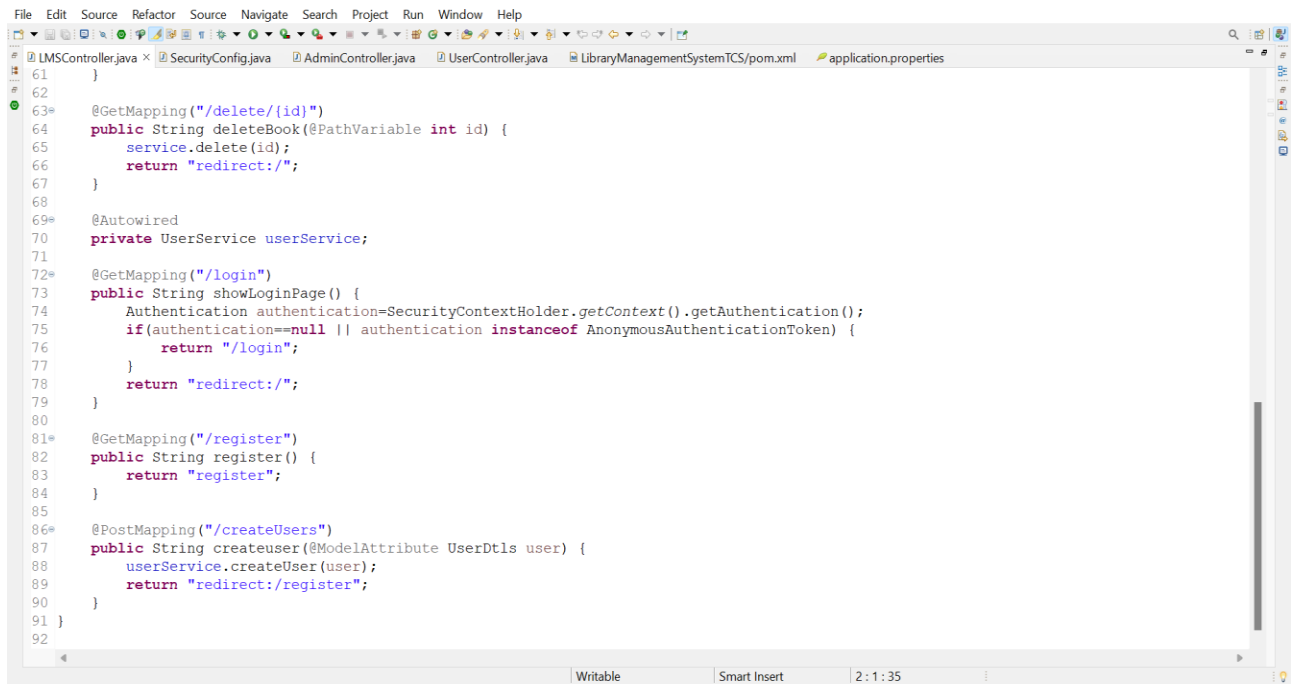
User Controller class



```
File Edit Source Refactor Source Navigate Search Project Run Window Help
1 package com.sayan.lms.controller;
2
3 import java.security.Principal;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.stereotype.Controller;
6 import org.springframework.ui.Model;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.ModelAttribute;
9 import org.springframework.web.bind.annotation.RequestMapping;
10
11 import com.sayan.lms.entity.*;
12 import com.sayan.lms.repository.*;
13
14 @Controller
15 @RequestMapping("/user")
16 public class UserController {
17     @Autowired
18     private UserRepository userRepo;
19
20     @ModelAttribute
21     private void userDetails(Model model, Principal p) {
22         String email = p.getName();
23         UserDtls user = userRepo.findByEmail(email);
24
25         model.addAttribute("user", user);
26     }
27
28     @GetMapping("/")
29     public String home() {
30         return "user";
31     }
32 }
```

Home Controller class

```
File Edit Source Refactor Source Navigate Search Project Run Window Help
LMSController.java x SecurityConfig.java AdminController.java UserController.java LibraryManagementSystemTCS/pom.xml application.properties
1 package com.sayan.lms.controller;
2
3 import java.util.List;
4
5 @Controller
6 public class LMSController {
7     @Autowired
8     private BookServices service;
9     @GetMapping("/")
10    public String home(Model m) {
11        List<Book> book=service.getAllBook();
12        m.addAttribute("book",book);
13        return "index";
14    }
15
16    @GetMapping("/user")
17    public String userHome(Model m) {
18        List<Book> book=service.getAllBook();
19        m.addAttribute("book",book);
20        return "userindex";
21    }
22
23    @GetMapping("/addbook")
24    public String addBookForm() {
25        return "add_book";
26    }
27
28    @PostMapping("/register")
29    public String bookRegister(@ModelAttribute Book b) {
30        service.addBook(b);
31        return "add_book";
32    }
33
34    @GetMapping("/edit/{id}")
35    public String editBook(@PathVariable int id,Model m) {
36        Book b=service.getBookById(id);
37        m.addAttribute("book", b);
38        return "edit";
39    }
40
41    @PostMapping("/update")
42    public String updateBook(@ModelAttribute Book b) {
43        service.addBook(b);
44        return "redirect:/";
45    }
46
47    @GetMapping("/delete/{id}")
48    public String deleteBook(@PathVariable int id) {
49        service.delete(id);
50        return "redirect:/";
51    }
52
53    @Autowired
54    private UserService userService;
55
56    @GetMapping("/login")
57    public String showLoginPage() {
58        Authentication authentication=SecurityContextHolder.getContext().getAuthentication();
59        if(authentication==null || authentication instanceof AnonymousAuthenticationToken) {
60            return "/login";
61        }
62        return "redirect:/";
63    }
64
65    @GetMapping("/logout")
66    public String logout() {
67        return "redirect:/";
68    }
69
70    @GetMapping("/about")
71    public String about() {
72        return "about";
73    }
74
75    @GetMapping("/contact")
76    public String contact() {
77        return "contact";
78    }
79
80    @GetMapping("/faq")
81    public String faq() {
82        return "faq";
83    }
84
85    @GetMapping("/terms")
86    public String terms() {
87        return "terms";
88    }
89
90    @GetMapping("/privacy")
91    public String privacy() {
92        return "privacy";
93    }
94
95    @GetMapping("/help")
96    public String help() {
97        return "help";
98    }
99
100    @GetMapping("/error")
101    public String error() {
102        return "error";
103    }
104
105    @GetMapping("/404")
106    public String notFound() {
107        return "404";
108    }
109
110    @GetMapping("/500")
111    public String serverError() {
112        return "500";
113    }
114
115    @GetMapping("/403")
116    public String forbidden() {
117        return "403";
118    }
119
120    @GetMapping("/401")
121    public String unauthorized() {
122        return "401";
123    }
124
125    @GetMapping("/400")
126    public String badRequest() {
127        return "400";
128    }
129
130    @GetMapping("/406")
131    public String notAcceptable() {
132        return "406";
133    }
134
135    @GetMapping("/408")
136    public String requestTimeout() {
137        return "408";
138    }
139
140    @GetMapping("/409")
141    public String conflict() {
142        return "409";
143    }
144
145    @GetMapping("/501")
146    public String notImplemented() {
147        return "501";
148    }
149
150    @GetMapping("/502")
151    public String badGateway() {
152        return "502";
153    }
154
155    @GetMapping("/503")
156    public String serviceUnavailable() {
157        return "503";
158    }
159
160    @GetMapping("/504")
161    public String gatewayTimeout() {
162        return "504";
163    }
164
165    @GetMapping("/505")
166    public String httpVersionNotSupported() {
167        return "505";
168    }
169
170    @GetMapping("/506")
171    public String variantNotNegotiated() {
172        return "506";
173    }
174
175    @GetMapping("/507")
176    public String requestEntityTooLarge() {
177        return "507";
178    }
179
180    @GetMapping("/508")
181    public String requestHeaderFieldsTooLarge() {
182        return "508";
183    }
184
185    @GetMapping("/509")
186    public String requestHeaderFieldsTooLarge() {
187        return "509";
188    }
189
190    @GetMapping("/510")
191    public String notExtended() {
192        return "510";
193    }
194
195    @GetMapping("/511")
196    public String networkAuthenticationRequired() {
197        return "511";
198    }
199
200    @GetMapping("/512")
201    public String networkAuthenticationRequired() {
202        return "512";
203    }
204
205    @GetMapping("/513")
206    public String networkAuthenticationRequired() {
207        return "513";
208    }
209
210    @GetMapping("/514")
211    public String networkAuthenticationRequired() {
212        return "514";
213    }
214
215    @GetMapping("/515")
216    public String networkAuthenticationRequired() {
217        return "515";
218    }
219
220    @GetMapping("/516")
221    public String networkAuthenticationRequired() {
222        return "516";
223    }
224
225    @GetMapping("/517")
226    public String networkAuthenticationRequired() {
227        return "517";
228    }
229
230    @GetMapping("/518")
231    public String networkAuthenticationRequired() {
232        return "518";
233    }
234
235    @GetMapping("/519")
236    public String networkAuthenticationRequired() {
237        return "519";
238    }
239
240    @GetMapping("/520")
241    public String networkAuthenticationRequired() {
242        return "520";
243    }
244
245    @GetMapping("/521")
246    public String networkAuthenticationRequired() {
247        return "521";
248    }
249
250    @GetMapping("/522")
249    public String networkAuthenticationRequired() {
250        return "522";
251    }
252
253    @GetMapping("/523")
254    public String networkAuthenticationRequired() {
255        return "523";
256    }
257
258    @GetMapping("/524")
259    public String networkAuthenticationRequired() {
260        return "524";
261    }
262
263    @GetMapping("/525")
264    public String networkAuthenticationRequired() {
265        return "525";
266    }
267
268    @GetMapping("/526")
269    public String networkAuthenticationRequired() {
270        return "526";
271    }
272
273    @GetMapping("/527")
274    public String networkAuthenticationRequired() {
275        return "527";
276    }
277
278    @GetMapping("/528")
279    public String networkAuthenticationRequired() {
280        return "528";
281    }
282
283    @GetMapping("/529")
284    public String networkAuthenticationRequired() {
285        return "529";
286    }
287
288    @GetMapping("/530")
289    public String networkAuthenticationRequired() {
290        return "530";
291    }
292
293    @GetMapping("/531")
294    public String networkAuthenticationRequired() {
295        return "531";
296    }
297
298    @GetMapping("/532")
299    public String networkAuthenticationRequired() {
300        return "532";
301    }
302
303    @GetMapping("/533")
304    public String networkAuthenticationRequired() {
305        return "533";
306    }
307
308    @GetMapping("/534")
309    public String networkAuthenticationRequired() {
310        return "534";
311    }
312
313    @GetMapping("/535")
314    public String networkAuthenticationRequired() {
315        return "535";
316    }
317
318    @GetMapping("/536")
319    public String networkAuthenticationRequired() {
320        return "536";
321    }
322
323    @GetMapping("/537")
324    public String networkAuthenticationRequired() {
325        return "537";
326    }
327
328    @GetMapping("/538")
329    public String networkAuthenticationRequired() {
330        return "538";
331    }
332
333    @GetMapping("/539")
334    public String networkAuthenticationRequired() {
335        return "539";
336    }
337
338    @GetMapping("/540")
339    public String networkAuthenticationRequired() {
340        return "540";
341    }
342
343    @GetMapping("/541")
344    public String networkAuthenticationRequired() {
345        return "541";
346    }
347
348    @GetMapping("/542")
349    public String networkAuthenticationRequired() {
350        return "542";
351    }
352
353    @GetMapping("/543")
354    public String networkAuthenticationRequired() {
355        return "543";
356    }
357
358    @GetMapping("/544")
359    public String networkAuthenticationRequired() {
360        return "544";
361    }
362
363    @GetMapping("/545")
364    public String networkAuthenticationRequired() {
365        return "545";
366    }
367
368    @GetMapping("/546")
369    public String networkAuthenticationRequired() {
370        return "546";
371    }
372
373    @GetMapping("/547")
374    public String networkAuthenticationRequired() {
375        return "547";
376    }
377
378    @GetMapping("/548")
379    public String networkAuthenticationRequired() {
380        return "548";
381    }
382
383    @GetMapping("/549")
384    public String networkAuthenticationRequired() {
385        return "549";
386    }
387
388    @GetMapping("/550")
389    public String networkAuthenticationRequired() {
390        return "550";
391    }
392
393    @GetMapping("/551")
394    public String networkAuthenticationRequired() {
395        return "551";
396    }
397
398    @GetMapping("/552")
399    public String networkAuthenticationRequired() {
400        return "552";
401    }
402
403    @GetMapping("/553")
404    public String networkAuthenticationRequired() {
405        return "553";
406    }
407
408    @GetMapping("/554")
409    public String networkAuthenticationRequired() {
410        return "554";
411    }
412
413    @GetMapping("/555")
414    public String networkAuthenticationRequired() {
415        return "555";
416    }
417
418    @GetMapping("/556")
419    public String networkAuthenticationRequired() {
420        return "556";
421    }
422
423    @GetMapping("/557")
424    public String networkAuthenticationRequired() {
425        return "557";
426    }
427
428    @GetMapping("/558")
429    public String networkAuthenticationRequired() {
430        return "558";
431    }
432
433    @GetMapping("/559")
434    public String networkAuthenticationRequired() {
435        return "559";
436    }
437
438    @GetMapping("/560")
439    public String networkAuthenticationRequired() {
440        return "560";
441    }
442
443    @GetMapping("/561")
444    public String networkAuthenticationRequired() {
445        return "561";
446    }
447
448    @GetMapping("/562")
449    public String networkAuthenticationRequired() {
450        return "562";
451    }
452
453    @GetMapping("/563")
454    public String networkAuthenticationRequired() {
455        return "563";
456    }
457
458    @GetMapping("/564")
459    public String networkAuthenticationRequired() {
460        return "564";
461    }
462
463    @GetMapping("/565")
464    public String networkAuthenticationRequired() {
465        return "565";
466    }
467
468    @GetMapping("/566")
469    public String networkAuthenticationRequired() {
470        return "566";
471    }
472
473    @GetMapping("/567")
474    public String networkAuthenticationRequired() {
475        return "567";
476    }
477
478    @GetMapping("/568")
479    public String networkAuthenticationRequired() {
480        return "568";
481    }
482
483    @GetMapping("/569")
484    public String networkAuthenticationRequired() {
485        return "569";
486    }
487
488    @GetMapping("/570")
489    public String networkAuthenticationRequired() {
490        return "570";
491    }
492
493    @GetMapping("/571")
494    public String networkAuthenticationRequired() {
495        return "571";
496    }
497
498    @GetMapping("/572")
499    public String networkAuthenticationRequired() {
500        return "572";
501    }
502
499    @GetMapping("/573")
500    public String networkAuthenticationRequired() {
501        return "573";
502    }
503
504    @GetMapping("/574")
505    public String networkAuthenticationRequired() {
506        return "574";
507    }
508
509    @GetMapping("/575")
509    public String networkAuthenticationRequired() {
510        return "575";
511    }
512
513    @GetMapping("/576")
514    public String networkAuthenticationRequired() {
515        return "576";
516    }
517
518    @GetMapping("/577")
519    public String networkAuthenticationRequired() {
520        return "577";
521    }
522
523    @GetMapping("/578")
524    public String networkAuthenticationRequired() {
525        return "578";
526    }
527
528    @GetMapping("/579")
529    public String networkAuthenticationRequired() {
530        return "579";
531    }
532
533    @GetMapping("/580")
534    public String networkAuthenticationRequired() {
535        return "580";
536    }
537
538    @GetMapping("/581")
539    public String networkAuthenticationRequired() {
540        return "581";
541    }
542
543    @GetMapping("/582")
544    public String networkAuthenticationRequired() {
545        return "582";
546    }
547
548    @GetMapping("/583")
549    public String networkAuthenticationRequired() {
550        return "583";
551    }
552
553    @GetMapping("/584")
554    public String networkAuthenticationRequired() {
555        return "584";
556    }
557
558    @GetMapping("/585")
559    public String networkAuthenticationRequired() {
560        return "585";
561    }
562
563    @GetMapping("/586")
564    public String networkAuthenticationRequired() {
565        return "586";
566    }
567
568    @GetMapping("/587")
569    public String networkAuthenticationRequired() {
570        return "587";
571    }
572
573    @GetMapping("/588")
574    public String networkAuthenticationRequired() {
575        return "588";
576    }
577
578    @GetMapping("/589")
579    public String networkAuthenticationRequired() {
580        return "589";
581    }
582
583    @GetMapping("/590")
584    public String networkAuthenticationRequired() {
585        return "590";
586    }
587
588    @GetMapping("/591")
589    public String networkAuthenticationRequired() {
590        return "591";
591    }
592
593    @GetMapping("/592")
594    public String networkAuthenticationRequired() {
595        return "592";
596    }
597
598    @GetMapping("/593")
599    public String networkAuthenticationRequired() {
600        return "593";
601    }
602
599    @GetMapping("/594")
600    public String networkAuthenticationRequired() {
601        return "594";
602    }
603
604    @GetMapping("/595")
605    public String networkAuthenticationRequired() {
606        return "595";
607    }
608
609    @GetMapping("/596")
610    public String networkAuthenticationRequired() {
611        return "596";
612    }
613
614    @GetMapping("/597")
615    public String networkAuthenticationRequired() {
616        return "597";
617    }
618
619    @GetMapping("/598")
620    public String networkAuthenticationRequired() {
621        return "598";
622    }
623
624    @GetMapping("/599")
625    public String networkAuthenticationRequired() {
626        return "599";
627    }
628
629    @GetMapping("/600")
630    public String networkAuthenticationRequired() {
631        return "600";
632    }
633
634    @GetMapping("/601")
635    public String networkAuthenticationRequired() {
636        return "601";
637    }
638
639    @GetMapping("/602")
640    public String networkAuthenticationRequired() {
641        return "602";
642    }
643
644    @GetMapping("/603")
645    public String networkAuthenticationRequired() {
646        return "603";
647    }
648
649    @GetMapping("/604")
650    public String networkAuthenticationRequired() {
651        return "604";
652    }
653
654    @GetMapping("/605")
655    public String networkAuthenticationRequired() {
656        return "605";
657    }
658
659    @GetMapping("/606")
660    public String networkAuthenticationRequired() {
661        return "606";
662    }
663
664    @GetMapping("/607")
665    public String networkAuthenticationRequired() {
666        return "607";
667    }
668
669    @GetMapping("/608")
670    public String networkAuthenticationRequired() {
671        return "608";
672    }
673
674    @GetMapping("/609")
675    public String networkAuthenticationRequired() {
676        return "609";
677    }
678
679    @GetMapping("/610")
680    public String networkAuthenticationRequired() {
681        return "610";
682    }
683
684    @GetMapping("/611")
685    public String networkAuthenticationRequired() {
686        return "611";
687    }
688
689    @GetMapping("/612")
690    public String networkAuthenticationRequired() {
691        return "612";
692    }
693
694    @GetMapping("/613")
695    public String networkAuthenticationRequired() {
696        return "613";
697    }
698
699    @GetMapping("/614")
700    public String networkAuthenticationRequired() {
701        return "614";
702    }
703
704    @GetMapping("/615")
705    public String networkAuthenticationRequired() {
706        return "615";
707    }
708
709    @GetMapping("/616")
710    public String networkAuthenticationRequired() {
711        return "616";
712    }
713
714    @GetMapping("/617")
715    public String networkAuthenticationRequired() {
716        return "617";
717    }
718
719    @GetMapping("/618")
720    public String networkAuthenticationRequired() {
721        return "618";
722    }
723
724    @GetMapping("/619")
725    public String networkAuthenticationRequired() {
726        return "619";
727    }
728
729    @GetMapping("/620")
730    public String networkAuthenticationRequired() {
731        return "620";
732    }
733
734    @GetMapping("/621")
735    public String networkAuthenticationRequired() {
736        return "621";
737    }
738
739    @GetMapping("/622")
740    public String networkAuthenticationRequired() {
741        return "622";
742    }
743
744    @GetMapping("/623")
745    public String networkAuthenticationRequired() {
746        return "623";
747    }
748
749    @GetMapping("/624")
750    public String networkAuthenticationRequired() {
751        return "624";
752    }
753
754    @GetMapping("/625")
755    public String networkAuthenticationRequired() {
756        return "625";
757    }
758
759    @GetMapping("/626")
760    public String networkAuthenticationRequired() {
761        return "626";
762    }
763
764    @GetMapping("/627")
765    public String networkAuthenticationRequired() {
766        return "627";
767    }
768
769    @GetMapping("/628")
770    public String networkAuthenticationRequired() {
771        return "628";
772    }
773
774    @GetMapping("/629")
775    public String networkAuthenticationRequired() {
776        return "629";
777    }
778
779    @GetMapping("/630")
780    public String networkAuthenticationRequired() {
781        return "630";
782    }
783
784    @GetMapping("/631")
785    public String networkAuthenticationRequired() {
786        return "631";
787    }
788
789    @GetMapping("/632")
790    public String networkAuthenticationRequired() {
791        return "632";
792    }
793
794    @GetMapping("/633")
795    public String networkAuthenticationRequired() {
796        return "633";
797    }
798
799    @GetMapping("/634")
800    public String networkAuthenticationRequired() {
801        return "634";
802    }
803
804    @GetMapping("/635")
805    public String networkAuthenticationRequired() {
806        return "635";
807    }
808
809    @GetMapping("/636")
810    public String networkAuthenticationRequired() {
811        return "636";
812    }
813
814    @GetMapping("/637")
815    public String networkAuthenticationRequired() {
816        return "637";
817    }
818
819    @GetMapping("/638")
820    public String networkAuthenticationRequired() {
821        return "638";
822    }
823
824    @GetMapping("/639")
825    public String networkAuthenticationRequired() {
826        return "639";
827    }
828
829    @GetMapping("/640")
830    public String networkAuthenticationRequired() {
831        return "640";
832    }
833
834    @GetMapping("/641")
835    public String networkAuthenticationRequired() {
836        return "641";
837    }
838
839    @GetMapping("/642")
840    public String networkAuthenticationRequired() {
841        return "642";
842    }
843
844    @GetMapping("/643")
845    public String networkAuthenticationRequired() {
846        return "643";
847    }
848
849    @GetMapping("/644")
850    public String networkAuthenticationRequired() {
851        return "644";
852    }
853
854    @GetMapping("/645")
855    public String networkAuthenticationRequired() {
856        return "645";
857    }
858
859    @GetMapping("/646")
860    public String networkAuthenticationRequired() {
861        return "646";
862    }
863
864    @GetMapping("/647")
865    public String networkAuthenticationRequired() {
866        return "647";
867    }
868
869    @GetMapping("/648")
870    public String networkAuthenticationRequired() {
871        return "648";
872    }
873
874    @GetMapping("/649")
875    public String networkAuthenticationRequired() {
876        return "649";
877    }
878
879    @GetMapping("/650")
880    public String networkAuthenticationRequired() {
881        return "650";
882    }
883
884    @GetMapping("/651")
885    public String networkAuthenticationRequired() {
886        return "651";
887    }
888
889    @GetMapping("/652")
890    public String networkAuthenticationRequired() {
891        return "652";
892    }
893
894    @GetMapping("/653")
895    public String networkAuthenticationRequired() {
896        return "653";
897    }
898
899    @GetMapping("/654")
900    public String networkAuthenticationRequired() {
901        return "654";
902    }
903
904    @GetMapping("/655")
905    public String networkAuthenticationRequired() {
906        return "655";
907    }
908
909    @GetMapping("/656")
910    public String networkAuthenticationRequired() {
911        return "656";
912    }
913
914    @GetMapping("/657")
915    public String networkAuthenticationRequired() {
916        return "657";
917    }
918
919    @GetMapping("/658")
920    public String networkAuthenticationRequired() {
921        return "658";
922    }
923
924    @GetMapping("/659")
925    public String networkAuthenticationRequired() {
926        return "659";
927    }
928
929    @GetMapping("/660")
930    public String networkAuthenticationRequired() {
931        return "660";
932    }
933
934    @GetMapping("/661")
935    public String networkAuthenticationRequired() {
936        return "661";
937    }
938
939    @GetMapping("/662")
940    public String networkAuthenticationRequired() {
941        return "662";
942    }
943
944    @GetMapping("/663")
945    public String networkAuthenticationRequired() {
946        return "663";
947    }
948
949    @GetMapping("/664")
950    public String networkAuthenticationRequired() {
951        return "664";
952    }
953
954    @GetMapping("/665")
955    public String networkAuthenticationRequired() {
956        return "665";
957    }
958
959    @GetMapping("/666")
960    public String networkAuthenticationRequired() {
961        return "666";
962    }
963
964    @GetMapping("/667")
965    public String networkAuthenticationRequired() {
966        return "667";
967    }
968
969    @GetMapping("/668")
970    public String networkAuthenticationRequired() {
971        return "668";
972    }
973
974    @GetMapping("/669")
975    public String networkAuthenticationRequired() {
976        return "669";
977    }
978
979    @GetMapping("/670")
980    public String networkAuthenticationRequired() {
981        return "670";
982    }
983
984    @GetMapping("/671")
985    public String networkAuthenticationRequired() {
986        return "671";
987    }
988
989    @GetMapping("/672")
990    public String networkAuthenticationRequired() {
991        return "672";
992    }
993
994    @GetMapping("/673")
995    public String networkAuthenticationRequired() {
996        return "673";
997    }
998
999    @GetMapping("/674")
1000    public String networkAuthenticationRequired() {
1001        return "674";
1002    }
1003
1004    @GetMapping("/675")
1005    public String networkAuthenticationRequired() {
1006        return "675";
1007    }
1008
1009    @GetMapping("/676")
1010    public String networkAuthenticationRequired() {
1011        return "676";
1012    }
1013
1014    @GetMapping("/677")
1015    public String networkAuthenticationRequired() {
1016        return "677";
1017    }
1018
1019    @GetMapping("/678")
1020    public String networkAuthenticationRequired() {
1021        return "678";
1022    }
1023
1024    @GetMapping("/679")
1025    public String networkAuthenticationRequired() {
1026        return "679";
1027    }
1028
1029    @GetMapping("/680")
1030    public String networkAuthenticationRequired() {
1031        return "680";
1032    }
1033
1034    @GetMapping("/681")
1035    public String networkAuthenticationRequired() {
1036        return "681";
1037    }
1038
1039    @GetMapping("/682")
1040    public String networkAuthenticationRequired() {
1041        return "682";
1042    }
1043
1044    @GetMapping("/683")
1045    public String networkAuthenticationRequired() {
1046        return "683";
1047    }
1048
1049    @GetMapping("/684")
1050    public String networkAuthenticationRequired() {
1051        return "684";
1052    }
1053
1054    @GetMapping("/685")
1055    public String networkAuthenticationRequired() {
1056        return "685";
1057    }
1058
1059    @GetMapping("/686")
1060    public String networkAuthenticationRequired() {
1061        return "686";
1062    }
1063
1064    @GetMapping("/687")
1065    public String networkAuthenticationRequired() {
1066        return "687";
1067    }
1068
1069    @GetMapping("/688")
1070    public String networkAuthenticationRequired() {
1071        return "688";
1072    }
1073
1074    @GetMapping("/689")
1075    public String networkAuthenticationRequired() {
1076        return "689";
1077    }
1078
1079    @GetMapping("/690")
1080    public String networkAuthenticationRequired() {
1081        return "690";
1082    }
1083
1084    @GetMapping("/691")
1085    public String networkAuthenticationRequired() {
1086        return "691";
1087    }
1088
1089    @GetMapping("/692")
1090    public String networkAuthenticationRequired() {
1091        return "692";
1092    }
1093
1094    @GetMapping("/693")
1095    public String networkAuthenticationRequired() {
1096        return "693";
1097    }
1098
1099    @GetMapping("/694")
1100    public String networkAuthenticationRequired() {
1101        return "694";
1102    }
1103
1104    @GetMapping("/695")
1105    public String networkAuthenticationRequired() {
1106        return "695";
1107    }
1108
1109    @GetMapping("/696")
1110    public String networkAuthenticationRequired() {
1111        return "696";
1112    }
1113
1114    @GetMapping("/697")
1115    public String networkAuthenticationRequired() {
1116        return "697";
1117    }
1118
1119    @GetMapping("/698")
1120    public String networkAuthenticationRequired() {
1121        return "698";
1122    }
1123
1124    @GetMapping("/699")
1125    public String networkAuthenticationRequired() {
1126        return "699";
1127    }
1128
1129    @GetMapping("/700")
1130    public String networkAuthenticationRequired() {
1131        return "700";
1132    }
1133
1134    @GetMapping("/701")
1135    public String networkAuthenticationRequired() {
1136        return "701";
1137    }
1138
1139    @GetMapping("/702")
1140    public String networkAuthenticationRequired() {
1141        return "702";
1142    }
1143
1144    @GetMapping("/703")
1145    public String networkAuthenticationRequired() {
1146        return "703";
1147    }
1148
1149    @GetMapping("/704")
1150    public String networkAuthenticationRequired() {
1151        return "704";
1152    }
1153
1154    @GetMapping("/705")
1155    public String networkAuthenticationRequired() {
1156        return "705";
1157    }
1158
1159    @GetMapping("/706")
1160    public String networkAuthenticationRequired() {
1161        return "706";
1162    }
1163
1164    @GetMapping("/707")
1165    public String networkAuthenticationRequired() {
1166        return "707";
1167    }
1168
1169    @GetMapping("/708")
1170    public String networkAuthenticationRequired() {
1171        return "708";
1172    }
1173
1174    @GetMapping("/709")
1175    public String networkAuthenticationRequired() {
1176        return "709";
1177    }
1178
1179    @GetMapping("/710")
1180    public String networkAuthenticationRequired() {
1181        return "710";
1182    }
1183
1184    @GetMapping("/711")
1185    public String networkAuthenticationRequired() {
1186        return "711";
1187    }
1188
1189    @GetMapping("/712")
1190    public String networkAuthenticationRequired() {
1191        return "712";
1192    }
1193
1194    @GetMapping("/713")
1195    public String networkAuthenticationRequired() {
1196        return "713";
1197    }
1198
1199    @GetMapping("/714")
1200    public String networkAuthenticationRequired() {
1201        return "714";
1202    }
1203
1204    @GetMapping("/715")
1205    public String networkAuthenticationRequired() {
1206        return "715";
1207    }
1208
1209    @GetMapping("/716")
1210    public String networkAuthenticationRequired() {
1211        return "716";
1212    }
1213
1214    @GetMapping("/717")
1215    public String networkAuthenticationRequired() {
1216        return "717";
1217    }
1218
1219    @GetMapping("/718")
1220    public String networkAuthenticationRequired() {
1221        return "718";
1222    }
1223
1224    @GetMapping("/719")
1225    public String networkAuthenticationRequired() {
1226        return "719";
1227    }
1228
1229    @GetMapping("/720")
1230    public String networkAuthenticationRequired() {
1231        return "720";
1232    }
1233
1234    @GetMapping("/721")
1235    public String networkAuthenticationRequired() {
1236        return "721";
1237    }
1238
1239    @GetMapping("/722")
1240    public String networkAuthenticationRequired() {
1241        return "722";
1242    }
1243
1244    @GetMapping("/723")
1245    public String networkAuthenticationRequired() {
1246        return "723";
1247    }
1248
1249    @GetMapping("/724")
1250    public String networkAuthenticationRequired() {
1251        return "724";
1252    }
1253
1254    @GetMapping("/725")
1255    public String networkAuthenticationRequired() {
1256        return "725";
1257    }
1258
1259    @GetMapping("/726")
1260    public String networkAuthenticationRequired() {
1261        return "726";
1262    }
1263
1264    @GetMapping("/727")
1265    public String networkAuthenticationRequired() {
1266        return "727";
1267    }
1268
1269    @GetMapping("/728")
1270    public String networkAuthenticationRequired() {
1271        return "728";
1272    }
1273
1274    @GetMapping("/729")
1275    public String networkAuthenticationRequired() {
1276        return "729";
1277    }
1278
1279    @GetMapping("/730")
1280    public String networkAuthenticationRequired() {
1281        return "730";
1282    }
1283
1284    @GetMapping("/731")
1285    public String networkAuthenticationRequired() {
1286        return "731";
1287    }
1288
1289    @GetMapping("/732")
1290    public String networkAuthenticationRequired() {
1291        return "732";
1292    }
1293
1294    @GetMapping("/733")
1295    public String networkAuthenticationRequired() {
1296        return "733";
1297    }
1298
1299    @GetMapping("/734")
1300    public String networkAuthenticationRequired() {
1301        return "734";
1302    }
1303
1304    @GetMapping("/735")
1305    public String networkAuthenticationRequired() {
1306        return "735";
1307    }
1308
1309    @GetMapping("/736")
1310    public String networkAuthenticationRequired() {
1311        return "736";
1312    }
1313
1314    @GetMapping("/737")
1315    public String networkAuthenticationRequired() {
1316        return "737";
1317    }
1318
1319    @GetMapping("/738")
1320    public String networkAuthenticationRequired() {
1321        return "738";
1322    }
1323
1324    @GetMapping("/739")
1325    public String networkAuthenticationRequired() {
1326        return "739";
1327    }
1328
1329    @GetMapping("/740")
1330    public String networkAuthenticationRequired() {
1331        return "740";
13
```



```
File Edit Source Refactor Source Navigate Search Project Run Window Help
LMSController.java x SecurityConfig.java AdminController.java UserController.java LibraryManagementSystemTCS/pom.xml application.properties
61 }
62
63 @GetMapping("/delete/{id}")
64 public String deleteBook(@PathVariable int id) {
65     service.delete(id);
66     return "redirect:/";
67 }
68
69 @Autowired
70 private UserService userService;
71
72 @GetMapping("/login")
73 public String showLoginPage() {
74     Authentication authentication=SecurityContextHolder.getContext().getAuthentication();
75     if(authentication==null || authentication instanceof AnonymousAuthenticationToken) {
76         return "/login";
77     }
78     return "redirect:/";
79 }
80
81 @GetMapping("/register")
82 public String register() {
83     return "register";
84 }
85
86 @PostMapping("/createUsers")
87 public String createuser(@ModelAttribute UserDtls user) {
88     userService.createUser(user);
89     return "redirect:/register";
90 }
91 }
92
Writable Smart Insert 2:1:35
```

SYSTEM TESTING

The aim of the system testing process was to determine all defects in our project. The program was subjected to a set of test inputs and various observations were made and based on these observations it will be decided whether the program behaves as expected or not. Our Project went through two levels of testing –

1. Unit testing
2. Integration testing

5.1 UNIT TESTING

Unit testing is undertaken when a module has been created and successfully reviewed. In order to test a single module we need to provide a complete environment i.e. besides the module we would require

- The procedures belonging to other modules that the module under test calls
- Non local data structures that module accesses
- A procedure to call the functions of the module under test with appropriate parameters

1. Test For the admin module
 - a. Testing admin login form -This form is used for log in of all the user. In this we enter the email and password if both are correct and the role of that user is “ADMIN” then admin dashboard will open otherwise if any of data is wrong it will get redirected back to the login page and again ask for email and password.
 - b. Add Book - In this section the admin can add new books by providing all the required details and the link to access the book.
 - c. Edit Book - Admin can edit the details of the existing books and save it.
 - d. Delete Book – Admin can delete a particular book from the list of existing books.

2. Test for normal user login module
 - a. Test for user login form - This form is used for log in of all the user. In this we enter the email and password, if all these are correct and role of the user is "USER" the user dashboard will open otherwise if any of data is wrong it will get redirected back to the login page and again ask for email and password.
 - b. Test for registration - This form is used for new account creation. A new user can register himself by providing the username, email and password. If user with the same email already exists then it will show user already exists. Then user need to use other email and register. After successful registration user will be able to login using the email and password in the login form.

5.2 INTEGRATION TESTING

In this type of testing, we test various integration of the project module by providing the input. The primary objective is to test the module interfaces in order to ensure that no errors are occurring when one module invokes the other module.

CONCLUSION & FUTURE SCOPE

This website provides a computerized version of library management system which will benefit the user as well as the staff of the library. It makes entire process online where user can find all the books, admin can add new books and edit or, delete existing books. It also has a facility for user login where user can login himself and can see all available books and read them using the link associated with it. There is a future scope of this facility that many more features such as online lectures video tutorials can be added by admin as well as online assignments submission facility, a feature of group chat where users can discuss various issues of engineering can be added to this project thus making it more interactive more user friendly and project which fulfils each users need in the best way possible.

REFERENCES

1. <https://www.youtube.com/watch?v=s4jh8lVtTvo&t=3072s>
2. https://www.w3schools.com/html/html_intro.asp
3. https://www.w3schools.com/js/js_datatypes.asp
4. <https://www.w3schools.com/bootstrap>
5. <https://www.daitm.org.in/wp-content/uploads/2019/04/Gr.-06library-project-report.pdf>